

CloudCampus

Multi-Tenant SaaS School Management Platform

Enterprise Documentation

Infrastructure · Architecture · Monitoring · Operations · Security

Version:	1.0.0 -- Production Release
Platform:	Spring Boot 3.4 · Java 21 · React · React Native
Database:	PostgreSQL 16 + Redis + RabbitMQ
Tenancy:	Multi-Tenant SaaS · Enterprise-Grade
Prepared for:	Customer Demos · Investors · Engineering
Generated:	17 May 2026

CONFIDENTIAL -- For authorized personnel only

1. Platform Overview

CloudCampus is an enterprise-grade, multi-tenant SaaS School Management Platform built on Spring Boot 3.4 and Java 21. It provides a complete digital backbone for K-12 schools, from admissions and academics to finance, communication, and AI-powered insights.

Core Value Proposition

- Multi-tenant architecture -- one platform, unlimited schools
- Enterprise-grade security and compliance

Module Map

Module	Key Features
Authentication	JWT · Refresh tokens · RBAC · Multi-device sessions
Academic	Classes · Sections · Timetable · Lesson Plans · Exams
Attendance	QR-code · Manual · Staff attendance · Analytics
Finance	Fee structures · UPI/Razorpay · Receipts · Defaulters
Communication	Email · SMS · WhatsApp · Push · Notice board
AI Gateway	Claude + GPT · Prompt registry · Usage analytics
Monitoring	Prometheus · Grafana · Tempo · JVM · DB metrics
Mobile	React Native · Offline sync · Biometric auth

2. Architecture Overview

CloudCampus follows a layered, modular monolith architecture running on Docker Compose for local development and Kubernetes-ready containers for production. Each bounded context is a Java package with its own controller ? service ? repository stack.

Technology Stack

Layer	Technology	Version
Backend API	Spring Boot	3.4.5
Language	Java	21 LTS
ORM	Hibernate / Spring JPA	6.6
Database	PostgreSQL	16
Cache	Redis	7.2
Message Broker	RabbitMQ	3.13
Object Storage	MinIO	RELEASE.2024
Migrations	Flyway	10.20
Frontend	React + TypeScript	18 / Vite 5
Mobile	React Native	0.74
Observability	Prometheus + Grafana	2.x / 10.x
Tracing	Grafana Tempo	2.x
AI	Spring AI	1.0.0

Request Flow

Request lifecycle:

```
Browser/Mobile ? Nginx (TLS termination)
? Spring Boot API (Port 8080)
  ? JWT Filter ? Tenant Filter ? Rate Limiter
  ? Controller ? Service ? Repository (JPA)
  ? PostgreSQL (primary data)
  ? Redis (session cache, rate-limit counters)
  ? RabbitMQ (async notifications)
  ? MinIO (file uploads)
? Prometheus scrape /actuator/prometheus
? Grafana dashboard / Tempo trace
```

3. Multi-Tenant Architecture

CloudCampus uses shared-schema multi-tenancy. Every tenant-scoped table carries a `tenant_id` UUID column. Hibernate `@Filter` ensures every JPA query automatically appends `WHERE tenant_id = :currentTenant`. Cross-tenant data leakage is impossible at the ORM layer.

Isolation Layers

- JWT claim `tenant_id` -- extracted on every request, stored in `RequestContext`

- Hibernate

Tenant Provisioning Flow

Tenant onboarding sequence:

1. SuperAdmin calls `POST /v1/super-admin/tenants`
2. `TenantServiceImpl` creates: `tenants` row + `schools` row + `MAIN` school
3. Flyway baseline schema already present (shared)
4. Default features seeded from `features` table
5. Subscription record created (BASIC by default)
6. SuperAdmin enables features / upgrades plan as needed
7. SchoolAdmin invited ? JWT includes `tenant_id` + `school_id`

Demo Tenant

Field	Value
Tenant ID	c0000000-0000-0000-0000-000000000001
Slug	greenwood-demo
School	Greenwood International School
Plan	Enterprise (all features enabled)
Mode	Read-only demo -- writes blocked via <code>DemoModelInterceptor</code>
Admin login	gw.admin / Demo@1234

4. School Setup Flow

A new school on CloudCampus goes through a structured onboarding flow that provisions all required data automatically. The system is designed for zero-touch provisioning.

Onboarding Checklist

Step	Action	Automated?
1	Create Tenant record	Yes -- API
2	Provision MAIN school	Yes -- TenantServiceImpl
3	Seed feature flags	Yes -- from features table
4	Create academic year	Manual -- SchoolAdmin
5	Define classes & sections	Manual -- SchoolAdmin
6	Import teacher roster	Manual -- CSV upload
7	Import student roster	Manual -- CSV upload
8	Configure fee structures	Manual -- Finance module
9	Set timetable	Manual -- Timetable module
10	Enable communication channels	Config -- API keys

5. Demo Tenant Strategy

Greenwood International School is the platform's permanent enterprise showcase tenant. It is seeded automatically on startup when `app.demo.enabled=true` and reset nightly at 02:00 AM server time.

Data Summary

Module	Demo Data
Academic Year	2025-26 (current)
Classes	15 (Nursery, LKG, UKG, Class 1-12)
Sections	45 (3 per class: A, B, C)
Subjects	10 (Math, Science, English, Hindi, SST, CS, Phy, Chem, Bio, PE)
Students	1,130 (25 per section) + 5 named demo students
Teachers	40 + 1 school admin
Attendance	20 working days, 90% present rate
Exams	2 (Unit Test 1, Mid-Term) with marks & results
Lesson Plans	10 (PUBLISHED)
Homework	3 assignments
Assignments	5 (with student submissions)
Notices	5 school notices (published)
Timetable	3 sections × 6 days × 8 periods = 144 slots
Staff Attendance	15 staff × 20 days
Leave Requests	10 (APPROVED / REJECTED / PENDING)
Fee Records	Tuition + Exam fee per demo student
Notifications	15 email/SMS logs
WhatsApp Logs	10 WhatsApp message logs
AI Usage Logs	15 AI call records (Claude + GPT)

Demo Credentials

Role	Username	Password	Portal
School Admin	gw.admin	Demo@1234	/school-admin/dashboard
Teacher 1	gw.teacher001	Demo@1234	/teacher/dashboard
Teacher 2	gw.teacher002	Demo@1234	/teacher/dashboard
Student	gw.student001	Demo@1234	/student/dashboard
Parent	gw.parent001	Demo@1234	/parent/dashboard

6. Seeder Architecture

The demo seeder is an `ApplicationRunner` that executes once at startup. It is conditionally activated via `@ConditionalOnProperty` and guarded against re-seeding.

Key Classes

Class	Purpose
DemoDataSeeder	<code>ApplicationRunner @Order(20)</code> -- seeds all demo data
DemoConstants	Stable UUID constants + shared configuration
IndianNameGenerator	Deterministic Indian names, phones, addresses (seed=42M)
DemoModelInterceptor	<code>HandlerInterceptor</code> -- blocks writes for demo JWT
DemoResetScheduler	<code>@Scheduled(0 0 2 * * *)</code> -- nightly data reset

Idempotent Guard

Startup guard logic:

```
SELECT COUNT(*) FROM students WHERE tenant_id = DEMO_TENANT_ID
-- If count >= 100 ? skip seeding (already done)
-- If count < 100 ? run full seed (first boot or after manual wipe)
```

Seeder Execution Order

```
@Order(10) -- SuperAdminBootstrap (superadmin user)
@Order(20) -- DemoDataSeeder (all Greenwood demo data)
```

Seed sequence within `DemoDataSeeder`:

1. Academic year
2. Departments + fee categories
3. Subjects (10)
4. Classes (15) + Sections (45)
5. Admin user + staff record
6. 40 teacher users + staff records
7. 5 named demo student + parent users
8. 1,130 bulk students
9. Fee structures
10. Attendance (6 sections × 20 days)
11. Exams + marks + results
12. Lesson plans (10)
13. Homework (3)
14. Notices (5)
15. Timetable (144 slots)
16. Staff attendance (15 staff × 20 days)
17. Leave requests (10)
18. Student fee records + payments
19. Assignments (5 + submissions)
20. Notification logs (15)
21. WhatsApp logs (10)
22. AI usage logs (15)

7. Docker Setup

CloudCampus infrastructure runs entirely in Docker Compose. The backend and frontend run locally during development but are containerised for staging and production.

Container Inventory

Service	Image	Port	Purpose
postgres	postgres:16-alpine	5432	Primary database
redis	redis:7.2-alpine	6379	Cache + rate limiting
rabbitmq	rabbitmq:3.13-management	5672/15672	Message broker + mgmt UI
minio	minio/minio	9000/9001	Object storage + console
mailhog	mailhog/mailhog	1025/8025	Dev SMTP + web UI
grafana	grafana/grafana	3100	Metrics dashboards
prometheus	prom/prometheus	9090	Metrics scraping
tempo	grafana/tempo	3200	Distributed tracing

Start / Stop Commands

```
# Start all infrastructure
docker compose up -d postgres redis rabbitmq minio mailhog grafana prometheus tempo

# Start backend (from /backend directory)
mvn spring-boot:run -Dspring-boot.run.profiles=dev

# Start frontend (from /frontend directory)
npm run dev

# Stop everything
docker compose down

# Full reset (including volumes)
docker compose down -v
```


8. PostgreSQL Setup

Connection Details (Dev)

Parameter	Value
Host	localhost
Port	5432
Database	cloudcampus
Username	cloudcampus
Password	cloudcampus
JDBC URL	jdbc:postgresql://localhost:5432/cloudcampus

Schema Management -- Flyway

All schema changes are managed exclusively through Flyway versioned migrations in backend/src/main/resources/db/migration/. Never modify the schema manually.

```
# Current migration count
SELECT version, description, installed_on
FROM flyway_schema_history
ORDER BY installed_rank DESC LIMIT 10;

# Check pending migrations
mvn flyway:info -pl backend

# Repair checksum mismatches
mvn flyway:repair -pl backend
```

Key Tables

- tenants -- top-level tenant registry

- schools

9. Redis Setup

Redis serves two functions: session/JWT caching and distributed rate-limit counters.

Configuration

```
# application-dev.yml
spring:
  data:
    redis:
      host: localhost
      port: 6379
      password:          # empty in dev
      timeout: 2000ms
      lettuce:
        pool:
          max-active: 10
          max-idle: 5
```

Key Patterns

Key Pattern	Purpose	TTL
rate:login:{ip}	Login brute-force counter	15 min
rate:api:{userId}	Per-user API rate limit	1 min
session:{jti}	Invalidated JWT JTI blacklist	Token expiry
refresh:{token}	Refresh token validity	7 days
tenant:features:{tid}	Feature flag cache	30 min

Monitoring

```
# Connect to Redis CLI
docker exec -it cloudcampus-redis-1 redis-cli

# Memory usage
INFO memory

# Key count
DBSIZE

# Slow log
SLOWLOG GET 10

# Monitor live commands
MONITOR
```

10. RabbitMQ Setup

RabbitMQ handles all async notifications: email, SMS, WhatsApp, and push.

Connection Details

Parameter	Value
Host	localhost
AMQP Port	5672
Management UI	http://localhost:15672
Username	guest
Password	guest

Queues

Queue	Purpose	DLQ
notification.email	Outbound email dispatch	notification.email.dlq
notification.sms	Outbound SMS dispatch	notification.sms.dlq
notification.whatsapp	WhatsApp Business API messages	notification.wa.dlq
notification.push	FCM push notifications	notification.push.dlq
audit.events	Async audit log writes	--

```
# List queues with message counts
docker exec cloudcampus-rabbitmq-1 \
  rabbitmqctl list_queues name messages consumers

# Purge a stuck queue (dev only)
docker exec cloudcampus-rabbitmq-1 \
  rabbitmqctl purge_queue notification.email
```

11. MinIO Setup

MinIO provides S3-compatible object storage for student documents, profile photos, assignment files, and generated PDF reports.

Connection

Parameter	Value
Endpoint	http://localhost:9000
Console	http://localhost:9001
Access Key	minioadmin
Secret Key	minioadmin

Buckets

Bucket	Contents	Access
cloudcampus-docs	Student documents, ID cards	Private
cloudcampus-media	Profile photos, school logos	Public read
cloudcampus-reports	Generated PDF reports	Private
cloudcampus-backup	Database dump archives	Private

```
# MinIO health check
curl http://localhost:9000/minio/health/live

# List buckets via mc (MinIO client)
mc alias set local http://localhost:9000 minioadmin minioadmin
mc ls local/
```

12. Frontend Setup

Tech Stack

Technology	Version	Purpose
React	18	UI framework
TypeScript	5.x	Type safety
Vite	5.x	Build tool / dev server
React Query	5.x	Server state + caching
Zustand	4.x	Client state management
React Router	6.x	SPA routing
Tailwind CSS	3.x	Utility-first styling

Dev Setup

```
cd frontend
npm install
npm run dev          # http://localhost:5173

# Production build
npm run build
npm run preview
```

Feature Structure

```
frontend/src/
  features/
    auth/          -- login, token refresh
    school-admin/  -- dashboard, students, fees, notices
    teacher/       -- lesson plans, attendance, exams
    student/       -- results, timetable, homework
    parent/        -- fees, attendance alerts
    super-admin/   -- tenant management
  common/         -- shared components, hooks, utils
  stores/         -- Zustand auth + tenant store
```

13. Mobile Setup

CloudCampus Mobile is a React Native app covering parent and student portals with offline sync and biometric authentication.

Key Features

- Parent portal: attendance alerts, fee payments, report cards

- Student

Development

```
cd mobile
npm install

# iOS
cd ios && pod install && cd ..
npx react-native run-ios

# Android
npx react-native run-android
```

14. AI Module Setup

CloudCampus AI Gateway (CC-1600) provides managed access to Claude and GPT models with token tracking, cost attribution per tenant, and a prompt template registry.

Supported Models

Provider	Model	Use Case
Anthropic	claude-sonnet-4-6	Lesson plans, report cards, risk analysis
Anthropic	claude-haiku-4-5	Quick summaries, homework feedback
OpenAI	gpt-4o-mini	Attendance insights, fee alerts

Prompt Registry

Prompt templates are stored in ai_prompt_templates with versioning. Only one version per prompt_key can be is_active=true (enforced by partial unique index).

AI Features

- lesson_plan_generator -- AI-generated weekly lesson plans from topic + grade
- student_

Configuration

```
# application-dev.yml
spring:
  ai:
    anthropic:
      api-key: ${ANTHROPIC_API_KEY}
      chat.options.model: claude-sonnet-4-6
    openai:
      api-key: ${OPENAI_API_KEY}
      chat.options.model: gpt-4o-mini
```

15. Feature Flag System

Every CloudCampus capability is gated by a feature flag stored in `tenant_features`. This enables per-tenant plan enforcement and gradual rollout.

Architecture

<code>features</code> table	-- global catalogue of feature keys
<code>tenant_features</code> table	-- (tenant_id, feature_key, enabled)
<code>TenantFeatureService</code>	-- checks cache ? DB ? denies if disabled
Redis cache	-- 30-min TTL per tenant feature set

Key Feature Keys

Feature Key	Description	Plan
attendance	Student attendance module	BASIC+
fee_management	Fee collection + receipts	BASIC+
timetable	Timetable management	STANDARD+
ai_lesson_plans	AI lesson plan generation	ENTERPRISE
ai_risk_analysis	AI student risk prediction	ENTERPRISE
custom_domain	Custom DNS domain	ENTERPRISE
whatsapp	WhatsApp Business API	STANDARD+
hostel	Hostel management	STANDARD+
transport	Transport module	STANDARD+
website_builder	School website builder	ENTERPRISE

16. Subscription System

Plan	Student Limit	Features
BASIC	Up to 300	Core modules: attendance, fees, notices
STANDARD	Up to 1,000	BASIC + timetable, WhatsApp, hostel, transport
ENTERPRISE	Unlimited	All features + AI + custom domain + API access

Enforcement

SubscriptionService checks the current plan tier before allowing module access. Upgrade/downgrade is handled by SuperAdmin via `/v1/super-admin/tenants/{id}/subscription`.

17. Branding System

Enterprise tenants can white-label CloudCampus with custom domains, logos, primary colors, and school-specific email sender names.

Capabilities

- Custom domain -- e.g., erp.greenwood-intl.edu.in (DNS TXT verification)

- School I

18. Authentication Flow

Login + token refresh flow:

```
POST /v1/auth/login
{ username, password }
? AuthController ? AuthServiceImpl
? UserRepository.findByUsername()
? BCryptPasswordEncoder.matches()
? AuditLogService.record(LOGIN)
? JwtUtil.generateAccessToken()    TTL: 15 min
? JwtUtil.generateRefreshToken()  TTL: 7 days
? { accessToken, refreshToken, role, tenantId, schoolId }
```



```
POST /v1/auth/refresh
{ refreshToken }
? Validate refresh token signature + expiry
? Rotate refresh token (old invalidated in Redis)
? { accessToken, refreshToken }
```

JWT Claims

Access token payload:

```
{
  "sub":      "<userId>",
  "jti":      "<unique token ID>",
  "tenant_id": "<tenantId>",
  "school_id": "<schoolId>",
  "role":     "SCHOOL_ADMIN | TEACHER | STUDENT | PARENT",
  "iat":      "<issued-at epoch>",
  "exp":      "<expiry epoch>"
}
```

19. JWT Security

- Algorithm: HMAC-SHA256 (HS256) -- secret from environment variable JWT_SECRET

- Access

CRITICAL: Never set JWT_SECRET to a static value in code. Always use an environment variable or Vault secret.

Token Revocation

```
# Invalidate all sessions for a user (e.g., password change)
POST /v1/auth/logout
  ? Adds JTI to Redis blacklist with TTL = remaining token lifetime
  ? Deletes all refresh tokens for user

# Force-logout all sessions (admin action)
POST /v1/super-admin/users/{userId}/invalidate-sessions
```

20. Demo Reset System

DemoResetScheduler runs nightly at 02:00 AM. It deletes transient data, preserves structural data, and re-seeds all content via DemoDataSeeder.

Reset Schedule

```
@Scheduled(cron = "0 0 2 * * *")
@ConditionalOnProperty(name = "app.demo.enabled", havingValue = "true")
public class DemoResetScheduler {
    // Deletes: marks, results, exam_subjects, exams, attendance,
    //          lesson_plans, homework, notices, bulk students
    // Preserves: tenant, school, classes, sections, subjects,
    //            named demo students (GW-0001 to GW-0005)
    // Re-seeds:  calls DemoDataSeeder.run(null)
}
```

Manual Force Reset

```
# Wipe students to trigger full re-seed on next restart
DELETE FROM students
WHERE tenant_id = 'c0000000-0000-0000-0000-000000000001'
  AND student_number NOT IN ('GW-0001', 'GW-0002', 'GW-0003', 'GW-0004', 'GW-0005');

# Then restart backend:
pkill -f CloudCampusApplication && mvn spring-boot:run -pl backend
```

21. CI/CD Pipeline

Recommended Pipeline Stages

Stage	Tool	Action
Build	Maven	mvn clean package -DskipTests
Unit Tests	JUnit 5	mvn test
Integration	Testcontainers	Docker Postgres + Redis in CI
Code Quality	SonarQube	sonar:sonar analysis
Docker Build	Docker	docker build -t cloudcampus:\$VERSION .
Push Image	ECR/GCR	docker push registry/cloudcampus:\$VERSION
Deploy	Helm/k8s	helm upgrade cloudcampus ./charts/
Smoke Test	curl	GET /actuator/health ? UP

Environment Promotion

dev ? staging ? production

dev: auto-deploy on merge to main

staging: auto-deploy on Git tag (v*.*.*-rc)

production: manual approval gate ? deploy on Git tag (v*.*.*)

22. Environment Variables

Variable	Required	Description
DB_URL	Yes	JDBC URL for PostgreSQL
DB_USERNAME	Yes	PostgreSQL username
DB_PASSWORD	Yes	PostgreSQL password
REDIS_HOST	Yes	Redis hostname
REDIS_PASSWORD	Prod	Redis AUTH password
RABBITMQ_HOST	Yes	RabbitMQ hostname
RABBITMQ_USERNAME	Yes	RabbitMQ username
RABBITMQ_PASSWORD	Yes	RabbitMQ password
JWT_SECRET	Yes	HS256 signing secret (min 32 chars)
MINIO_ENDPOINT	Yes	MinIO endpoint URL
MINIO_ACCESS_KEY	Yes	MinIO access key
MINIO_SECRET_KEY	Yes	MinIO secret key
ANTHROPIC_API_KEY	AI	Anthropic Claude API key
OPENAI_API_KEY	AI	OpenAI GPT API key
RAZORPAY_KEY_ID	Pay	Razorpay key ID
RAZORPAY_KEY_SECRET	Pay	Razorpay key secret
app.demo.enabled	Dev	true = enable Greenwood demo seeder

23. Production Deployment

Docker Production Compose

```
# Production uses separate docker-compose.prod.yml
# Key differences from dev:
# - No mailhog (real SMTP)
# - Postgres with replication
# - Redis with AUTH password
# - RabbitMQ with TLS
# - MinIO behind Nginx reverse proxy
# - All secrets from Docker secrets / Vault

docker compose -f docker-compose.prod.yml up -d
```

Kubernetes (Production)

```
# Namespace
kubectl create namespace cloudcampus

# Deploy via Helm
helm install cloudcampus ./charts/cloudcampus \
  --namespace cloudcampus \
  --set image.tag=1.0.0 \
  --set postgres.external=true \
  --set secrets.jwtSecret=$JWT_SECRET

# Check rollout
kubectl rollout status deployment/cloudcampus -n cloudcampus
```


24. Monitoring & Observability

CloudCampus exposes a rich set of metrics via Micrometer, scraped by Prometheus, visualised in Grafana, and traced via Tempo (OpenTelemetry).

Observability Stack

Component	URL (Dev)	Purpose
Prometheus	http://localhost:9090	Metrics storage & alerting
Grafana	http://localhost:3100	Dashboards & visualization
Tempo	http://localhost:3200	Distributed trace storage
Actuator	http://localhost:8080/actuator	Health + metrics endpoint
RabbitMQ UI	http://localhost:15672	Queue monitoring
MinIO Console	http://localhost:9001	Storage monitoring

Key Metrics

- http_server_requests_seconds -- API response time histogram

- hikaricp

25. Backup & Restore

PostgreSQL Backup (pgbackup)

```
# The pgbackup sidecar runs pg_dump daily at 03:00 AM
# Dumps are stored in MinIO: cloudcampus-backup/postgres/YYYY-MM-DD/

# Manual backup
docker exec cloudcampus-postgres-1 \
  pg_dump -U cloudcampus cloudcampus | gzip > backup_$(date +%Y%m%d).sql.gz

# Restore
gunzip -c backup_20260101.sql.gz | \
  docker exec -i cloudcampus-postgres-1 \
  psql -U cloudcampus cloudcampus
```

Redis Backup

```
# Redis RDB snapshot
docker exec cloudcampus-redis-1 redis-cli BGSAVE

# Copy dump.rdb
docker cp cloudcampus-redis-1:/data/dump.rdb ./redis_backup_$(date +%Y%m%d).rdb
```

26. Scaling Strategy

Vertical Scaling (Current)

Single-node Docker Compose deployment handles up to ~50 concurrent tenants and ~5,000 daily active users comfortably on a 4-core / 8 GB VM.

Horizontal Scaling (>50 tenants)

Component	Scale Strategy
Backend API	Stateless -- run N replicas behind load balancer
PostgreSQL	Read replica for reporting queries
Redis	Redis Cluster with 3 shards
RabbitMQ	Clustered 3-node quorum queues
MinIO	Distributed mode across 4 nodes
Grafana	Single instance (monitoring only)

IMPORTANT: The backend is already stateless (no in-memory session state). Horizontal scaling requires only a shared DB, Redis, and RabbitMQ cluster.

27. Customer Demo Flow

A typical 30-minute sales demo covering all key modules.

Time	Module	What to Show
0:00-3:00	Login + Dashboard	gw.admin login, real-time stats cards
3:00-7:00	Students	1,130 students, search/filter, profile view
7:00-11:00	Attendance	Attendance history, 90% rate, analytics chart
11:00-15:00	Finance	Fee records, PAID/PARTIAL/PENDING, receipts
15:00-18:00	Exams & Results	Unit Test 1 marks, grade distribution
18:00-21:00	AI Features	Lesson plan generation, risk analyzer
21:00-24:00	Timetable	Class 1-A weekly schedule
24:00-27:00	Communication	Notification logs, WhatsApp messages
27:00-30:00	Monitoring	Grafana dashboard, container health

TIP: All write operations return HTTP 403 in demo mode -- this prevents accidental data corruption during live demos.

28. Read-Only Protections

DemoModelInterceptor intercepts all HTTP requests before they reach the controller. It reads the `tenant_id` claim from the JWT and blocks POST/PUT/PATCH/DELETE for the demo tenant.

Interceptor Logic

```
if (method is GET or HEAD) ? allow
if (URI starts with /v1/auth/ or /v1/public/ or /actuator/) ? allow
Extract JWT from Authorization: Bearer <token>
Decode payload (no signature verify -- already done upstream)
If tenant_id == DEMO_TENANT_ID:
    return HTTP 403 { code: DEMO_READ_ONLY }
else:
    allow
```

Error Response

HTTP 403 response body:

```
{
  "success": false,
  "error": {
    "code": "DEMO_READ_ONLY",
    "message": "This is a read-only demo environment. Write operations are disabled."
  }
}
```

29. Infrastructure Monitoring

Real-time container health monitored via docker stats and Prometheus node exporter.

Container Resource Usage (Typical Dev)

Container	RAM Usage	CPU%	Status
cloudcampus (backend)	~877 MB	2-8%	Healthy
postgres	~60 MB	0.5-2%	Healthy
redis	~19 MB	0.1%	Healthy
rabbitmq	~158 MB	0.5%	Healthy
grafana	~175 MB	0.5%	Healthy
prometheus	~92 MB	0.3%	Healthy
tempo	~177 MB	0.2%	Healthy
minio	~152 MB	0.2%	Healthy
mailhog	~16 MB	0.0%	Healthy

Total Infrastructure RAM: ~1.73 GB. Minimum recommended host: 8 GB RAM, 4 vCPUs.

30. Docker Troubleshooting

Essential Commands

```
# Real-time stats for all containers
docker stats --no-stream

# Check running containers
docker compose ps

# Follow logs for a container
docker compose logs -f postgres

# Inspect container details
docker inspect cloudcampus-postgres-1

# Execute shell in container
docker exec -it cloudcampus-postgres-1 bash

# Check container health
docker inspect --format='{{.State.Health.Status}}' cloudcampus-postgres-1

# View last 50 log lines
docker compose logs --tail=50 cloudcampus
```

Common Issues

Problem	Diagnosis	Fix
Port already in use	lsof -ti:5432	Kill conflicting process
Container OOM killed	docker inspect (OOMKilled)	Increase Docker memory limit
Postgres won't start	docker logs postgres	Check volume permissions
Redis timeout	redis-cli ping	Restart redis container
Flyway migration fail	Check flyway_schema_history	mvn flyway:repair

31. Container Health Checks

```
# Backend API
curl http://localhost:8080/actuator/health
# Expected: {"status":"UP"}

# PostgreSQL
docker exec cloudcampus-postgres-1 pg_isready -U cloudcampus
# Expected: localhost:5432 - accepting connections

# Redis
docker exec cloudcampus-redis-1 redis-cli ping
# Expected: PONG

# RabbitMQ
curl http://localhost:15672/api/aliveness-test/%%2F \
  -u guest:guest
# Expected: {"status":"ok"}

# MinIO
curl http://localhost:9000/minio/health/live
# Expected: HTTP 200

# Full stack health script
for svc in 8080 5432 6379 5672 9000; do
  nc -zv localhost $svc 2>&1 | grep -E 'succeeded|refused'
done
```


32. Memory Usage Analysis

Understanding Container Memory

All values below reflect typical usage after 30 minutes of warm operation with demo school data loaded. Values will be higher under real production load.

Container	RAM	Normal?	Notes
cloudcampus	877 MB	YES	Spring Boot + JVM heap. High because JVM pre-allocates.
postgres	60 MB	YES	Low -- shared_buffers defaults are conservative.
redis	19 MB	YES	Very low -- only holds session cache keys.
rabbitmq	158 MB	YES	Erlang VM baseline. Normal for empty queues.
grafana	175 MB	YES	Node.js backend. Increases with more dashboards.
prometheus	92 MB	YES	Grows with scrape history. ~500 MB at 2 weeks.
tempo	177 MB	YES	Trace storage buffer. Spikes during high traffic.
minio	152 MB	YES	Go binary. Stable unless many concurrent uploads.
mailhog	16 MB	YES	Very lightweight -- dev SMTP only.

Warning Signs

- Backend RAM > 2 GB ? possible heap leak, check JVM heap with jmap

- Postgres

33. JVM Monitoring

Key JVM Metrics via Actuator

```
# Heap memory
curl http://localhost:8080/actuator/metrics/jvm.memory.used

# GC pause time
curl http://localhost:8080/actuator/metrics/jvm.gc.pause

# Thread count
curl http://localhost:8080/actuator/metrics/jvm.threads.live

# CPU usage
curl http://localhost:8080/actuator/metrics/process.cpu.usage
```

JVM Heap Configuration

```
# Set in JAVA_TOOL_OPTIONS or application startup
export JAVA_TOOL_OPTIONS="-Xms512m -Xmx1024m -XX:+UseG1GC"

# For production (4 GB host)
JAVA_TOOL_OPTIONS="-Xms768m -Xmx1536m -XX:+UseG1GC \
-XX:MaxGCPauseMillis=200 \
-XX:+HeapDumpOnOutOfMemoryError \
-XX:HeapDumpPath=/dumps/heap.hprof"
```

Identify Memory Leaks

- Monitor jvm.memory.used over time in Grafana -- should stabilize after warm-up

- Continues

34. PostgreSQL Monitoring

```
-- Active connections
SELECT count(*), state FROM pg_stat_activity GROUP BY state;

-- Long-running queries (> 5 seconds)
SELECT pid, now()-pg_stat_activity.query_start AS duration, query
FROM pg_stat_activity
WHERE state = 'active'
      AND now()-query_start > interval '5 seconds'
ORDER BY duration DESC;

-- Table sizes
SELECT relname, pg_size_pretty(pg_total_relation_size(relid))
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC LIMIT 20;

-- Database size
SELECT pg_size_pretty(pg_database_size('cloudcampus'));

-- Index hit ratio (should be > 99%)
SELECT round(100*idx_blks_hit/(idx_blks_hit+idx_blks_read+0.001),2)
      AS index_hit_ratio
FROM pg_statio_user_tables
WHERE idx_blks_read > 0
LIMIT 10;
```

HikariCP Pool Metrics

Metric	Prometheus Key	Healthy Value
Active connections	hikaricp_connections_active	< pool max (10)
Idle connections	hikaricp_connections_idle	> 0
Pending threads	hikaricp_connections_pending	0
Connection timeout	hikaricp_connections_timeout	0

35. Redis Monitoring

```
docker exec cloudcampus-redis-1 redis-cli INFO memory

# Key metrics from INFO memory:
# used_memory_human      -- total allocated memory
# used_memory_rss_human  -- resident set size (OS view)
# maxmemory_human        -- configured limit (0 = unlimited)
# mem_fragmentation_ratio -- should be 1.0-1.5 (> 2.0 = fragmentation issue)

docker exec cloudcampus-redis-1 redis-cli INFO stats
# Key metrics:
# total_commands_processed
# instantaneous_ops_per_sec
# keyspace_hits / keyspace_misses (aim for > 95% hit rate)
# evicted_keys (if > 0 ? maxmemory policy kicking in)

# Monitor key expiry
docker exec cloudcampus-redis-1 redis-cli INFO keyspace
```

36. RabbitMQ Monitoring

```
# Queue depths (messages waiting)
docker exec cloudcampus-rabbitmq-1 \
  rabbitmqctl list_queues name messages consumers

# Expected output (healthy):
# notification.email      0    1
# notification.sms       0    1
# notification.whatsapp  0    1
# notification.push       0    1
# audit.events           0    1

# Node health
docker exec cloudcampus-rabbitmq-1 rabbitmqctl node_health_check

# Memory usage
docker exec cloudcampus-rabbitmq-1 rabbitmqctl status | grep memory
```

Alert Thresholds

Metric	Warning	Critical
Queue depth	> 100 msgs	> 1,000 msgs
Consumer count	0 consumers	--
Memory	> 300 MB	> 500 MB
Connection count	> 100	> 500

37. Prometheus Metrics

Scrape Configuration

```
# prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'cloudcampus-backend'
    static_configs:
      - targets: ['host.docker.internal:8080']
    metrics_path: /actuator/prometheus

  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres-exporter:9187']

  - job_name: 'redis'
    static_configs:
      - targets: ['redis-exporter:9121']
```

Useful PromQL Queries

```
# 95th percentile API response time
histogram_quantile(0.95,
  sum(rate(http_server_requests_seconds_bucket[5m])) by (le, uri)
)

# Error rate (5xx)
sum(rate(http_server_requests_seconds_count{status=~"5.."}[5m]))
/
sum(rate(http_server_requests_seconds_count[5m]))

# JVM heap used %
jvm_memory_used_bytes{area="heap"}
/
jvm_memory_max_bytes{area="heap"}
```

38. Grafana Dashboards

Access Grafana at <http://localhost:3100> | admin / admin

Recommended Dashboards

Dashboard	Import ID	Shows
Spring Boot JVM	12900	Heap, GC, threads, CPU
PostgreSQL	9628	Connections, queries, table size
Redis	763	Memory, ops/sec, hit rate
RabbitMQ	10991	Queues, messages, consumers
MinIO	13502	Bucket size, request rate
Node Exporter	1860	Host CPU, RAM, disk, network

CloudCampus Custom Panels

- Active tenants over time -- count(distinct tenant_id) from request metrics

- Request

Alerting

```
# Example Grafana alert rule (JSON API)
# Fires when P95 latency > 500ms for 5 consecutive minutes
alert: HighApiLatency
expr: |
  histogram_quantile(0.95,
    rate(http_server_requests_seconds_bucket[5m])
  ) > 0.5
for: 5m
labels:
  severity: warning
annotations:
  summary: API P95 latency above 500ms
```

39. Tempo Distributed Tracing

Every API request generates an OpenTelemetry trace automatically via Micrometer Tracing. Traces are stored in Grafana Tempo and linked from Grafana logs.

Configuration

```
# application.yml
management:
  tracing:
    sampling:
      probability: 1.0    # 100% in dev; use 0.1 in production

spring:
  application:
    name: cloudcampus-backend

# Tempo OTLP endpoint (via docker-compose)
OTEL_EXPORTER_OTLP_ENDPOINT: http://tempo:4317
```

What to Look For

- Trace ID in API response headers: X-B3-TraceId

- Slow sp

40. MinIO Storage Monitoring

```
# Storage usage per bucket
mc alias set local http://localhost:9000 minioadmin minioadmin
mc du local/

# List all buckets with sizes
mc ls --summarize local/

# MinIO health check
curl http://localhost:9000/minio/health/live
curl http://localhost:9000/minio/health/cluster

# Admin info (API)
mc admin info local
```

Capacity Planning

Data Type	Avg Size	1K Students/Year
Profile photos	150 KB	~150 MB
Student documents	500 KB	~500 MB
Assignment files	200 KB	~200 MB/term
Generated PDF reports	100 KB	~100 MB/term
DB backups (gzip)	50 MB	~1.8 GB/year (daily)

41. Capacity Planning

Scale	Tenants	Students	Host Spec	PostgreSQL	Redis
Small	1-10	1-5K	4 vCPU / 8 GB	Single	Single
Medium	10-50	5-50K	8 vCPU / 16 GB	Single+read	Single
Large	50-200	50-200K	16 vCPU / 32 GB	Primary+2R	Cluster
Enterprise	200+	200K+	K8s autoscale	RDS Multi-AZ	Elasticache

Database Growth Estimates

Per tenant with 1,000 students, 40 staff, and 1 academic year of data:

Table	Rows/year	Size
attendance_records	180,000	~50 MB
student_marks	2,000	~1 MB
fee_payments	12,000	~5 MB
notification_logs	50,000	~20 MB
ai_usage_logs	10,000	~5 MB
audit_logs	500,000	~200 MB

42. Production Best Practices

- Always use SSL/TLS -- Nginx terminates TLS before reaching Spring Boot

- Set JWT

43. Security Best Practices

- JWT validation on every non-public request -- never skip the filter chain

- Password

SECURITY REQUIREMENT: All communication between services inside Docker network must use internal hostnames. Never expose Postgres/Redis ports to internet.

44. SaaS Scaling Guidance

Scaling Triggers

Metric	Threshold	Action
API P95 latency	> 500 ms	Add backend replica
DB connections	> 80% pool	Add read replica or increase pool
Redis memory	> 80%	Add Redis cluster shard
RabbitMQ queue depth	> 500 msgs	Add consumer instance
MinIO storage	> 80%	Add MinIO volume
JVM heap	> 80%	Tune -Xmx or add instance

Multi-Region Strategy

- Deploy primary region (India -- ap-south-1) for low latency

- Replicat

45. Troubleshooting Guide

Symptom	Check	Fix
Backend won't start	nohup.out / maven log	Check Flyway migration errors
Login returns 401	JWT_SECRET matches across deploys	Restart with correct secret
403 on demo writes	Expected -- DemoModelInterceptor	Use non-demo credentials
500 INTERNAL_ERROR	backend logs correlation ID	grep correlation ID in logs
Slow API (> 2s)	Grafana Trace ? slow DB span	Add missing index
Redis connection refused	docker ps grep redis	docker compose up -d redis
RabbitMQ consumer lag	rabbitmqctl list_queues	Restart notification service
MinIO upload fails	curl minio/health/live	Check disk space on host
Flyway checksum mismatch	flyway_schema_history	mvn flyway:repair
OOM killed container	docker inspect (OOMKilled)	Increase Docker memory limit

46. Recovery Procedures

Database Corruption

1. Stop backend immediately
`pkill -f CloudCampusApplication`
2. Take a snapshot of current state
`docker exec cloudcampus-postgres-1 \`
`pg_dump -U cloudcampus cloudcampus > corrupt_snapshot.sql`
3. Restore from last clean backup
`docker exec -i cloudcampus-postgres-1 \`
`psql -U cloudcampus -c 'DROP DATABASE cloudcampus;'`
`docker exec -i cloudcampus-postgres-1 \`
`psql -U cloudcampus -c 'CREATE DATABASE cloudcampus;'`
`gunzip -c backup_YYYYMMDD.sql.gz | \`
`docker exec -i cloudcampus-postgres-1 \`
`psql -U cloudcampus cloudcampus`
4. Restart backend (Flyway repairs automatically)
`mvn spring-boot:run -pl backend`

Lost Demo Data

```
# Force demo reseed
docker exec cloudcampus-postgres-1 psql -U cloudcampus cloudcampus \
  -c "DELETE FROM students WHERE tenant_id='c0000000-0000-0000-0000-000000000001'"
# Then restart backend -- DemoDataSeeder will re-seed automatically
```

47. Alerting Architecture

Alert Channels

Channel	Tool	When
Email	Alertmanager ? SMTP	Non-urgent, business hours
Slack/Teams	Alertmanager ? webhook	Engineering team alerts
PagerDuty	Alertmanager ? PD	P1 -- requires immediate response
Grafana oncall	Grafana OnCall	On-call rotation management

Critical Alerts (P1 -- Page Immediately)

- Backend health check DOWN > 2 minutes

- PostgreSQL

Warning Alerts (P3 -- Slack notification)

- API P95 latency > 500ms for 10 minutes

- RabbitMQ

48. Backup Strategy

Component	Frequency	Retention	Storage	Restore Time
PostgreSQL	Daily (03:00 AM)	30 days	MinIO + S3	< 1 hour
Redis RDB	Hourly	7 days	MinIO	< 5 min
MinIO data	Daily replication	90 days	S3 Glacier	< 2 hours
Config files	Git + CI artifacts	Unlimited	GitHub	< 10 min

Backup Verification

Every backup must be tested monthly via restore drill. An automated restore test runs in a separate Docker environment on the 1st of each month.

```
# Automated backup verification (runs in CI monthly)
# 1. Download latest backup from MinIO
# 2. Restore to test PostgreSQL container
# 3. Run smoke test: SELECT COUNT(*) FROM tenants > 0
# 4. Report pass/fail to Slack
```

49. Disaster Recovery

Recovery Objectives

Scenario	RTO	RPO
Single container crash	< 1 min	0 (stateless)
Host machine failure	< 15 min	< 1 hour (last backup)
Data center outage	< 2 hours	< 1 hour
Ransomware / data loss	< 4 hours	< 24 hours

DR Runbook

Step 1 -- Declare incident, notify stakeholders
Step 2 -- Assess blast radius: which tenants affected?
Step 3 -- Provision replacement infrastructure (Terraform)
Step 4 -- Restore PostgreSQL from latest MinIO backup
Step 5 -- Restore Redis from RDB snapshot (or accept cold start)
Step 6 -- Deploy backend (Docker image from registry)
Step 7 -- Run Flyway migrations (idempotent -- safe to re-run)
Step 8 -- Run smoke tests: /actuator/health, login, student list
Step 9 -- Update DNS / load balancer to new IP
Step 10 -- Notify tenants, monitor for 30 minutes
Step 11 -- Post-mortem within 48 hours

50. Enterprise Deployment Checklist

Pre-Deployment

- ☐ All environment variables set in target environment

- ☐ JWT_

Deployment

- ☐ Blue-green deployment: bring up new version alongside old

- ☐ Health

Post-Deployment

- ☐ Monitor Grafana for 30 minutes -- no anomalies

- ☐ Error

CloudCampus is production-ready when all checklist items are ?. Never skip items under time pressure -- each protects against a real failure mode.

Section Index

1. Platform Overview	2
2. Architecture Overview	3
3. Multi-Tenant Architecture	4
4. School Setup Flow	5
5. Demo Tenant Strategy	6
6. Seeder Architecture	7
7. Docker Setup	8
8. PostgreSQL Setup	9
9. Redis Setup	10
10. RabbitMQ Setup	11
11. MinIO Setup	12
12. Frontend Setup	13
13. Mobile Setup	14
14. AI Module Setup	15
15. Feature Flag System	16
16. Subscription System	17
17. Branding System	18
18. Authentication Flow	19
19. JWT Security	20
20. Demo Reset System	21
21. CI/CD Pipeline	22
22. Environment Variables	23
23. Production Deployment	24
24. Monitoring & Observability	25
25. Backup & Restore	26
26. Scaling Strategy	27
27. Customer Demo Flow	28
28. Read-Only Protections	29
29. Infrastructure Monitoring	30
30. Docker Troubleshooting	31
31. Container Health Checks	32
32. Memory Usage Analysis	33
33. JVM Monitoring	34
34. PostgreSQL Monitoring	35
35. Redis Monitoring	36
36. RabbitMQ Monitoring	37
37. Prometheus Metrics	38
38. Grafana Dashboards	39
39. Tempo Distributed Tracing	40
40. MinIO Storage Monitoring	41
41. Capacity Planning	42
42. Production Best Practices	43

43. Security Best Practices	44
44. SaaS Scaling Guidance	45
45. Troubleshooting Guide	46
46. Recovery Procedures	47
47. Alerting Architecture	48
48. Backup Strategy	49
49. Disaster Recovery	50
50. Enterprise Deployment Checklist	51